



Department of Electronics & Communications

25SC2107E – Machine Learning

Skilling Experiment

Breast Cancer Wisconsin — Logistic Regression and Regularisation

Dataset: Breast Cancer Wisconsin | Course Outcome: CO2 | Environment: VS Code

Prepared by: **Dr. Omprakash Gottam**, Dept. of ECE, KLEF Hyderabad

1 Experiment : Breast Cancer Wisconsin — Logistic Regression and Regularisation

Dataset: Breast Cancer Wisconsin

Course Outcome: CO2

1.1 Aim

To build a logistic regression classifier that tells malignant tumours from benign ones, to deliberately create a situation where the model has more features than patients, and to show that regularisation rescues it. In Experiment- House Pricing Prediction– regularisation made almost no difference. Here it makes a large one, and the reason for the difference is the real lesson.

1.2 Learning Objectives

After completing this experiment, you should be able to:

- Explain why `LogisticRegression` uses `C` instead of `alpha`, and why the two work in *opposite* directions.
- Recognise overfitting by comparing training accuracy with test accuracy.
- Say when regularisation will help and when it will not, using the number of rows per feature.
- Use L1 to shrink a 495-feature model down to about a dozen features without losing accuracy.

1.3 Environment and Libraries

VS Code with the Python + Jupyter extensions, `.venv` activated.

```
pip install numpy pandas matplotlib scikit-learn
```

1.4 Dataset

Breast Cancer Wisconsin (Diagnostic). Each row is one tumour, described by measurements taken from a digitised image of a fine needle aspirate.

- 569 rows, 30 number columns, no missing values.
- **Answer column:** `target`, where **0 means malignant** (cancer) and **1 means benign** (not cancer). There are 212 malignant and 357 benign tumours.
- The 30 features are ten basic measurements — radius, texture, perimeter, area, smoothness, compactness, concavity, concave points, symmetry, fractal dimension — each reported three ways: the **mean**, the **error**, and the **worst** (largest) value found in the image.
- It ships inside `scikit-learn`, so `load_breast_cancer` needs no download at all.

Note

Why this experiment looks different from House Pricing Prediction

In House Pricing Prediction you swept `alpha` for Ridge and Lasso and the error barely moved. That was the honest answer for that data: 16,512 rows and only 8 features, so there was almost nothing to overfit.

Regularisation earns its keep in the opposite situation — **few rows, many features**. So in this experiment we build that situation on purpose: we train on only **100 patients**, and we expand the 30 measurements into **495 features**. With more features than patients, an unregularised model can memorise the training set perfectly and still be poor at new patients. That is exactly the disease regularisation cures.

1.5 Procedure

1. Create a notebook `skill_lab5.ipynb`.
2. Load the data and check how many tumours of each kind there are.
3. Work out the baseline accuracy you must beat.
4. Split off a deliberately small training set of 100 patients.
5. Expand the 30 features into 495 using `PolynomialFeatures`, then scale them.
6. Fit logistic regression with the penalty switched off and compare train against test.
7. Learn what `C` means and how it relates to `alpha`.
8. Plot training and test accuracy against `C` for the L2 penalty.
9. Do the same for L1, and count how many features survive.
10. List the features L1 chose to keep.
11. Compare confusion matrices and count the missed cancers.
12. Write `classification_benchmark()` to rank every model.

1.6 Notebook Implementation

Cell 1 — Import the libraries

```
1 import numpy as np
2 import pandas as pd
3 import matplotlib.pyplot as plt
4 from sklearn.datasets import load_breast_cancer
5 from sklearn.model_selection import train_test_split
6 from sklearn.preprocessing import StandardScaler, PolynomialFeatures
7 from sklearn.linear_model import LogisticRegression
8 from sklearn.metrics import accuracy_score, confusion_matrix
```

- Only *one* model is imported this time. In House Pricing Prediction the penalty came from a different class each time (`Ridge`, `Lasso`, `ElasticNet`). In logistic regression all three are settings *inside* `LogisticRegression`, chosen with the `penalty` argument.
- `PolynomialFeatures` is the new tool. It is what will turn 30 features into 495.

Cell 2 — Load the data

```
1 data = load_breast_cancer(as_frame=True)
2 cancer = data.frame
3 print(cancer.shape)
4 print(cancer["target"].value_counts())
5 print(data.target_names)
```

- 569 rows and 31 columns: the 30 measurements plus the answer.
- `target_names` prints `['malignant' 'benign']`. The order matters: it tells you **0 is malignant and 1 is benign**. This feels backwards, because 1 usually means “the thing we are looking for”. Check this on every medical dataset before you read a single result.

Cell 3 — The baseline you must beat

```
1 print("Always-benign accuracy:", round(cancer["target"].mean(), 4))
```

- 357 of 569 tumours are benign, so a lazy model that says “benign” every single time is right **62.7%** of the time. Any model that cannot beat 0.627 has learned nothing.
- That lazy model would also miss *every single cancer*, which is the worst possible mistake here. Accuracy alone will never show you that, which is why Cell 11 uses a confusion matrix.

Cell 4 — A deliberately small training set

```

1 X = cancer.drop(columns="target")
2 y = cancer["target"]
3
4 X_train, X_test, y_train, y_test = train_test_split(
5     X, y, train_size=100, random_state=42, stratify=y)
6 print(X_train.shape, X_test.shape)

```

- `train_size=100` keeps only 100 patients for training and leaves 469 for testing. That is backwards from normal practice, and it is on purpose: we want a small training set so that overfitting actually happens where you can see it.
- This is not as artificial as it sounds. Labelled medical data is genuinely scarce, because every label needs a biopsy and a pathologist.
- `stratify=y` keeps the same malignant-to-benign ratio in both halves. This matters far more with 100 rows than with 16,512: without it, an unlucky split could hand you very few malignant cases.

Cell 5 — Make 495 features out of 30

```

1 poly = PolynomialFeatures(degree=2, include_bias=False)
2 X_train_p = poly.fit_transform(X_train)
3 X_test_p = poly.transform(X_test)
4 print(X_train_p.shape, X_test_p.shape)
5
6 scaler = StandardScaler()
7 X_train_s = scaler.fit_transform(X_train_p)
8 X_test_s = scaler.transform(X_test_p)

```

- `PolynomialFeatures(degree=2)` keeps the original 30 features and adds every square (`radius2`) and every pair multiplied together (`radius × texture`). That gives $30 + 30 + 435 = 495$ features.
- Why do this at all? Because plain logistic regression can only draw a straight boundary. Products of features let it draw curved ones, which can genuinely help. The catch is the count.
- **Look hard at the shape: 100 rows, 495 columns.** There are now nearly five times as many features as patients. A model in this situation can always find some combination that separates the training data perfectly, even if that combination is pure coincidence.
- `fit.transform` on train, `transform` on test — for both tools, and in that order. Scaling is compulsory again, and doubly so now: a squared area term is astronomically larger than a smoothness term.

Cell 6 — What happens with no penalty at all

```

1 big = LogisticRegression(C=10000, max_iter=20000)
2 big.fit(X_train_s, y_train)
3
4 train_acc = accuracy_score(y_train, big.predict(X_train_s))
5 test_acc = accuracy_score(y_test, big.predict(X_test_s))
6 print("train:", round(train_acc, 4))
7 print("test :", round(test_acc, 4))
8 print("gap  :", round(train_acc - test_acc, 4))

```

- **Training accuracy is 1.0000. Perfect.** Every one of the 100 patients classified correctly. If you ever see a perfect training score, do not celebrate — get suspicious.
- Test accuracy is **0.9232**, which is 7.7 points lower. That difference is the **generalisation gap**, and it is the fingerprint of overfitting.

- The model has not learned what a tumour is. It has memorised 100 patients. With 495 knobs and 100 examples there is more than enough freedom to do that.
- `max_iter=20000` stops the convergence warning. With this many features the solver needs a lot of steps.

Note

C is not alpha. It is upside down.

In House Pricing Prediction, `Ridge(alpha=...)` took alpha, and **bigger alpha meant a stronger penalty**.

`LogisticRegression` takes C instead, and

$$C = \frac{1}{\alpha}$$

so **bigger C means a *weaker* penalty**. Everything is reversed.

You want	Ridge / Lasso	LogisticRegression
No penalty (overfits)	<code>alpha = 0</code>	<code>C = 10000</code> (very large)
A gentle penalty	small alpha	large C
A strong penalty	large alpha	small C
Everything wiped out	alpha huge	C tiny

This single inversion causes more confusion than anything else in this topic. If your accuracy-versus-C plot looks like the mirror image of what you expected, this is why. When you read that a model is “more regularised”, ask which knob is being turned.

Cell 7 — L2: accuracy against C

```

1 Cs = [0.001, 0.01, 0.1, 1, 10, 100, 1000]
2 l2_train, l2_test = [], []
3
4 for C in Cs:
5     model = LogisticRegression(penalty="l2", C=C, max_iter=20000)
6     model.fit(X_train_s, y_train)
7     l2_train.append(accuracy_score(y_train, model.predict(X_train_s)))
8     l2_test.append(accuracy_score(y_test, model.predict(X_test_s)))
9
10 plt.plot(Cs, l2_train, marker="o", label="train")
11 plt.plot(Cs, l2_test, marker="o", label="test")
12 plt.xscale("log")
13 plt.xlabel("C (larger = weaker penalty)")
14 plt.ylabel("accuracy")
15 plt.title("L2 penalty")
16 plt.legend()
17 plt.show()

```

- **Plot train and test on the same axes.** A test curve alone tells you which model is best; both curves together tell you *why*, and that is what you are here to learn.
- Read the picture right to left, from weak penalty to strong. On the right the two curves are far apart: train 1.000, test 0.923. Move left and the training curve comes down while the test curve goes *up*. They meet at `C = 0.1`, where test accuracy peaks at **0.9638**.
- **That is regularisation working: about 4 points of accuracy gained by making the model deliberately worse at the data it was trained on.**
- Push further left to `C = 0.001` and both curves fall together. The penalty is now so heavy the model cannot learn even the real patterns. Too much is as bad as too little.

Cell 8 — L1: accuracy against C, and how many features survive

```

1 l1_test, kept = [], []
2
3 for C in Cs:
4     model = LogisticRegression(penalty="l1", C=C, solver="liblinear",
5                               random_state=42, max_iter=20000)
6     model.fit(X_train_s, y_train)
7     l1_test.append(accuracy_score(y_test, model.predict(X_test_s)))
8     kept.append(int((model.coef_ != 0).sum()))
9
10 pd.DataFrame({"C": Cs, "test_acc": np.round(l1_test, 4),
11             "features_used": kept})

```

- `penalty="l1"` needs `solver="liblinear"` or `"saga"`. The default `lbfgs` solver cannot do L1 and will raise an error. This is the second most common mistake in this topic, after the `C` inversion.
- `random_state=42` matters here: `liblinear` is slightly random, so without a seed you get a different number of surviving features each run.
- Read the `features_used` column downwards — this is the whole point of L1. It goes **0, 0, 6, 13, 52, 117, 234** out of 495. The penalty is not just shrinking coefficients, it is switching features off completely.
- The best row is `C = 1`: **test accuracy 0.9659 using 13 features out of 495**. Better accuracy than the model that used all of them, from about one fortieth of the inputs.
- At `C = 0.01` every feature is gone and accuracy collapses to **0.3731**. With nothing left to work with, the model predicts “malignant” for everyone — and 0.3731 is simply the fraction of the test set that really is malignant.

Cell 9 — Which features did L1 keep?

```

1 best_l1 = LogisticRegression(penalty="l1", C=0.1, solver="liblinear",
2                             random_state=42, max_iter=20000)
3 best_l1.fit(X_train_s, y_train)
4
5 names = poly.get_feature_names_out(X.columns)
6 nz = np.flatnonzero(best_l1.coef_[0])
7 survivors = pd.DataFrame({"feature": names[nz], "coef":
8                          best_l1.coef_[0][nz]})
9 survivors.reindex(survivors.coef.abs().sort_values(ascending=False).index)

```

- Six features survive out of 495, and **every one of them is a product of two measurements**, not a plain measurement. So the feature expansion in Cell 5 was not wasted — the combinations really did carry more signal than the originals.
- The strongest is **worst radius × worst smoothness. worst concave points and worst texture** also keep appearing. These are exactly the measurements pathologists associate with malignancy, which is a good sign that the model found something real rather than noise.
- All six coefficients are negative. Remember 1 means *benign*, so a negative weight pushes towards **malignant**: bigger, rougher, more irregular tumours are more likely to be cancer. Always translate the sign back into the label meaning before you claim anything.
- **This is the practical reason to reach for L1**. A doctor will not act on a 495-term formula. A six-term one built from familiar measurements can at least be discussed.

Cell 10 — The mistakes that actually matter

```

1 best_l2 = LogisticRegression(penalty="l2", C=0.1, max_iter=20000)
2 best_l2.fit(X_train_s, y_train)

```

```

3
4 print("No penalty:\n", confusion_matrix(y_test, big.predict(X_test_s)))
5 print("L2, C=0.1:\n", confusion_matrix(y_test,
    best_l2.predict(X_test_s)))

```

- Rows are the truth, columns are the prediction, and row 0 is the malignant row. So the **top-right number is malignant tumours called benign** — a missed cancer, the error that actually hurts someone.
- Unregularised: **18 missed cancers**. Regularised: **12**. The regularised model also raises far fewer false alarms, 5 against 18.
- **So regularisation did not just move a decimal point in an accuracy score. It found 6 more cancers out of 469 patients.** That is what the 4-point gain in Cell 7 means once you translate it back into people.
- Never report accuracy alone for a medical classifier. Two models with identical accuracy can distribute their mistakes very differently, and here one kind of mistake is far worse than the other.

Deliverable Function

```

1 def classification_benchmark(X_train, y_train, X_test, y_test, Cs):
2     models = {"No penalty": LogisticRegression(penalty=None,
3         max_iter=20000)}
4
5     for C in Cs:
6         models["L2 C=" + str(C)] = LogisticRegression(
7             penalty="l2", C=C, max_iter=20000)
8         models["L1 C=" + str(C)] = LogisticRegression(
9             penalty="l1", C=C, solver="liblinear",
10             random_state=42, max_iter=20000)
11         models["ElasticNet C=" + str(C)] = LogisticRegression(
12             penalty="elasticnet", C=C, l1_ratio=0.5, solver="saga",
13             random_state=42, max_iter=20000)
14
15     rows = []
16     for name in models:
17         model = models[name]
18         model.fit(X_train, y_train)
19         train_acc = accuracy_score(y_train, model.predict(X_train))
20         test_acc = accuracy_score(y_test, model.predict(X_test))
21         rows.append({
22             "model": name,
23             "train_acc": train_acc,
24             "test_acc": test_acc,
25             "gap": train_acc - test_acc,
26             "features_used": int((model.coef_ != 0).sum()),
27         })
28
29     table = pd.DataFrame(rows)
30     table = table.sort_values("test_acc",
31         ascending=False).reset_index(drop=True)
32     return table.round(4)
33
34 results = classification_benchmark(X_train_s, y_train, X_test_s,
35     y_test,
36     [0.01, 0.1, 1, 100])

```

- Same pattern as House Pricing Prediction: a dictionary of named models, one loop, one dictionary of results per model, one DataFrame at the end. Only the contents changed, not the shape of the code.
- **Compare the top row with the No penalty row.** The winner is L1 C=1 at **0.9659** using **13** features; No penalty sits near the bottom at **0.9232** using all **495**. Better accuracy, 38 times fewer features.
- **Read the gap column as a lie detector.** Every model with `train_acc` of 1.00 has a gap of 0.05 or worse. Every good model has a gap near 0.02 and a training score slightly *below* perfect. A model that is not quite perfect on its own training data is usually the more trustworthy one.
- The two rows at the bottom with `features_used` = 0 are the over-penalised ones. Their gap is tiny, near 0.003 — **proof that a small gap alone is not a good sign.** A model that knows nothing is equally ignorant on both sets. Always read the gap next to the accuracy, never on its own.
- `penalty=None` is the modern spelling. Older tutorials use `penalty='none'` as a string, which is removed in current scikit-learn.

1.7 Expected Output

```
(569, 31)
target
1    357
0    212
['malignant', 'benign']
Always-benign accuracy: 0.6274
(100, 30) (469, 30)
(100, 495) (469, 495)

train: 1.0
test  : 0.9232
gap   : 0.0768

      C  test_acc  features_used      <- L1 sweep
0  0.001    0.3731             0
1  0.010    0.3731             0
2  0.100    0.9574             6
3  1.000    0.9659            13
4 10.000    0.9446            52
5 100.000    0.9296           117

No penalty:          L2, C=0.1:
[[157  18]          [[163  12]
 [ 18 276]]         [  5 289]]

      model  train_acc  test_acc   gap  features_used
0      L1 C=1        0.99    0.9659  0.0241           13
1      L2 C=0.1       0.99    0.9638  0.0262          495
2  ElasticNet C=1     0.99    0.9616  0.0284          134
3      L1 C=0.1       0.98    0.9574  0.0226             6
...
9      L2 C=100       1.00    0.9232  0.0768          495
10     No penalty     1.00    0.9232  0.0768          495
11 ElasticNet C=0.01   0.63    0.6269  0.0031             0
12      L1 C=0.01     0.37    0.3731 -0.0031             0
```


1.8 Result

Logistic regression was fitted to 100 training patients described by 495 polynomial features derived from the 30 Breast Cancer Wisconsin measurements. With the penalty effectively switched off the model reached perfect training accuracy of 1.000 but only 0.9232 on the 469 held-out patients, a generalisation gap of 7.7 points. An L2 penalty at $C = 0.1$ raised test accuracy to 0.9638 and narrowed the gap to 2.6 points. An L1 penalty at $C = 1$ did slightly better still, 0.9659, while using only 13 of the 495 features. Missed cancers fell from 18 to 12 and false alarms from 18 to 5. Over-penalising in the other direction ($C \leq 0.01$ with L1) eliminated every coefficient and collapsed accuracy to 0.3731.

Note

When does regularisation help?

- **House Pricing Prediction:** 16,512 rows, 8 features. About 2,000 rows per feature. Regularisation changed almost nothing.
- **Breast Cancer:** 100 rows, 495 features. About 0.2 rows per feature. Regularisation gained 4 points of accuracy and removed 97% of the features.

The rule of thumb: the fewer rows you have per feature, the more regularisation matters. When data is plentiful and features are few, the data itself keeps the model honest and you can skip the tuning. When features approach or outnumber rows — after a polynomial expansion, with text or gene data, or simply when labels are expensive — regularisation is not an optional refinement. It is the difference between a model that works and one that has only memorised.

And the three penalties behave exactly as they did in House Pricing Prediction: **L2** shrinks every coefficient but deletes none, **L1** sets coefficients to exactly zero and so selects features, and **ElasticNet** mixes the two through `l1_ratio`. The only thing that changed is the name of the knob: `C`, turning the opposite way.